Computers in Industry 172 (2025) 104350

Contents lists available at ScienceDirect

Computers in Industry

journal homepage: www.sciencedirect.com/journal/computers-in-industry

An agent-based approach for the automatic generation of valid SysMLv2 Models in industrial contexts

Eduardo Cibrián *, Jose Olivert-Iserte, Juan Llorens, Jose María Álvarez-Rodríguez

Computer Science and Engineering Department, Universidad Carlos III de Madrid, Av. de la Universidad, 30, Leganés, 28911, Madrid, Spain

ARTICLE INFO

Keywords:
Model-Based Systems Engineering (MBSE)
SysML v2
Large Language Models (LLMs)
Automated model generation
Agent-based systems
Retrieval-Augmented Generation (RAG)

ABSTRACT

Automating the generation of valid SysML v2 models from natural language specifications holds promise for advancing Model-Based Systems Engineering (MBSE) in industrial settings. However, current approaches based solely on Large Language Models (LLMs) often fail to meet the syntactic and semantic rigor required by formal modeling languages. This paper introduces a domain-informed, agent-based framework that combines LLMs with structured retrieval and iterative validation to synthesize correct SysML v2 models. The system integrates Retrieval-Augmented Generation (RAG) using a curated repository of SysML v2 examples and enforces compliance through a validation engine based on the official ANTLR grammar. Experimental results across diverse MBSE scenarios demonstrate that the integration of retrieval and validation mechanisms leads to a substantial improvement in model correctness and semantic alignment, beyond what each component achieves individually. This combined effect enables reliable, closed-loop generation of formal models from natural language, illustrating how domain-specific integration can transform general-purpose LLMs into reliable assistants for engineering design tasks.

1. Introduction

Model-Based Systems Engineering (MBSE) offers a structured approach addressing the complexity in modern system development through the use of formal models. Despite these advantages, the practical implementation of MBSE in industrial contexts remains challenging (Chami and Bruel, 2018), particularly as systems grow in scale and heterogeneity. The increasing demand for rapid system development cycles and the integration of multidisciplinary teams have increased the need for intelligent automation in model creation, validation, and evolution.

The recent evolution of SysML toward version 2 (SysML v2) marks a milestone in the Systems Engineering discipline. SysML v2 introduces a more expressive, formally defined syntax and semantics, and improved support for model consistency and reuse (Bajaj et al., 2022). However, these advancements also bring increased syntactic and semantic complexity (Kausch et al., 2025), which can hinder adoption and make more difficult the generation of correct models, especially in early design phases where requirements are often incomplete or inconsistent.

In parallel, recent advances in Artificial Intelligence (AI), notably in Large Language Models (LLMs), have demonstrated the potential for automating knowledge-intensive tasks such as code generation, requirements formalization, and document synthesis (Wang et al., 2023b; Pallagani et al., 2024). LLMs can translate natural language descriptions into structured artifacts, providing a promising approach for bridging the gap between non-formal requirements (usually text-based) and formal system models. Nevertheless, applying LLMs directly to SysML v2 model generation presents some challenges: LLMs may produce syntactically incorrect or semantically inconsistent models due to their limited understanding of domain-specific grammar and constraints, often requiring expert intervention and manual correction.

To address these limitations, this work proposes an agent-based framework integrating Retrieval-Augmented Generation (RAG) techniques and iterative syntactic and semantic validation for automated SysML v2 model synthesis from natural language specifications. The presented approach combines the generative capabilities of LLMs with a RAG strategy, leveraging a curated repository of SysML v2 model examples to provide relevant structural guidance. A key aspect of this approach is the integration of an automated validation engine, based on the official SysML v2 ANTLR (ANother Tool for Language Recognition) grammar which helps to ensure syntactic feedback to iteratively refine the generated models.

* Corresponding author.
E-mail addresses: ecibrian@inf.uc3m.es (E. Cibrián), jolivert@inf.uc3m.es (J. Olivert-Iserte), juan.llorens@uc3m.es (J. Llorens), jose.alvarez@uc3m.es (J.M. Álvarez-Rodríguez).

https://doi.org/10.1016/j.compind.2025.104350
Received 11 June 2025; Received in revised form 12 August 2025; Accepted 13 August 2025
Available online 24 August 2025

The contributions of this work are threefold: (1) the design and application of an intelligent agent architecture that combines LLMs, RAG, and formal validation of model synthesis; (2) a demonstration of how retrieval-augmented prompts based on real SysML v2 examples can substantially improve the correctness and relevance of the generated models; and (3) an empirical evaluation showcasing the effectiveness of the proposed approach in generating valid SysML v2 models from natural language requirements.

By automating model synthesis and validation, this research overcomes a key limitation in the adoption of MBSE, while also providing new insights for the development of expert systems that support complex engineering workflows. The proposed methodology establishes a foundation for future intelligent, model-driven engineering environments, in which human expertise and AI work in synergy to accelerate innovation and ensure system quality in industrial contexts.

2. State of the art

This section reviews the existing works relevant to the automated generation and validation of systems models, focusing on three key areas: the application of LLMs to code and model generation, existing techniques for model validation in MBSE, and the use of agent-based systems for automatic task execution and refinement.

2.1. LLMs for code and model generation

LLMs have demonstrated advanced capabilities in generating code for different programming languages and platforms (Ghaemi et al., 2024). For instance, AlphaCode has demonstrated proficiency in generating functional code from natural language descriptions (Li et al., 2022). Recent systematic evaluations have also demonstrated that code-specific LLMs, such as CodeLlama and CodeGeeX, often outperform general-purpose LLMs on standard code generation benchmarks, highlighting the importance of domain-adapted training and evaluation strategies (Liu et al., 2024a). However, a key limitation of these approaches is the potential for generating syntactically incorrect code, requiring manual debugging and correction. Furthermore, ensuring the semantic correctness and adherence to specific coding standards remains a challenge (Wang et al., 2023b). This has led to many research works into techniques for improving the reliability and trustworthiness of LLM-generated code, including fine-tuning (Chen et al., 2021), reinforcement learning (Ziegler et al., 2019), and incorporating formal verification methods. Advanced prompting techniques, such as chain-of-thought and retrieval-augmented prompting, as well as iterative self-critique and correction, have been shown to significantly improve the reliability of LLM-generated code (Sun et al., 2023; Dou et al., 2023). Recent works have extended these ideas by combining intelligent agents with LLMs to tackle specialized translation problems, such as converting natural language queries to SQL (Ojuri et al., 2025) or transforming user interface designs into HTML code using chain-of-thought reasoning (Yuan et al., 2025).

The application of LLMs to Model-Based Systems Engineering (MBSE) and SysML model generation is a relatively recent area of research that is gaining momentum (Bonner et al., 2024; DeHart et al., 2024). While the potential for automation is significant, a key challenge lies in ensuring the validity of the generated or modified artifacts, particularly with regards to syntactic correctness, semantic coherence, and consistency across different model views. Research works are exploring different ways to leverage LLMs beyond the direct generation of specific diagram types. For instance, recognizing the more human-readable nature of SysML v2, DeHart et al. (2024) investigates the use of LLMs primarily as an interpretive layer to enable conversational interaction with system models with the aim of simplifying manual intervention and reducing the need for deep technical expertise. Nevertheless, regardless of whether the focus is on direct generation or facilitated interaction, the fundamental challenge of guaranteeing model integrity and correctness still persists, demanding robust validation mechanisms within any LLM-driven MBSE workflow.

Recent studies have also explored the integration of LLMs with domain-specific ontologies and knowledge graphs to enhance the semantic grounding of generated models, thereby reducing the risk of producing artifacts that are syntactically correct but semantically invalid (Xu et al., 2023).

2.2. Model validation in MBSE

Ensuring the correctness and consistency of system models is the key to boost MBSE (Smith et al., 2023; Hecht and Chen, 2022)-based methods. Model validation and verification (V&V) are recognized as essential processes to avoid costly integration failures, incomplete requirement tracing, and invalid analysis results, which can lead to significant project overruns and delays (Hecht and Chen, 2022; Visure Solutions, 2024). Traditional validation approaches include manual reviews by domain experts, peer inspections, and the use of specialized modeling tools that often incorporate built-in consistency checkers or linters (Jones et al., 2019; Lee et al., 2020; Nguyen et al., 2022). These tool-based checkers can detect common errors like unconnected ports, type mismatches, or constraint violations, but their capabilities are often limited to the specific rules implemented by the tool vendor and may not cover the full scope of the SysML standard or custom domain constraints.

Formal verification methods, such as model checking, can provide stronger guarantees than traditional validation but are typically applied to specific behavioral aspects (e.g. state machines) (Brown et al., 2021) rather than the entire structural model, and often require significant expertise to set up and interpret. Recent research in the context of SysML v2 highlights how the introduction of formal semantics enables the application of advanced formal verification techniques, including automated reasoning and model checking to both behavioral and structural aspects of system models (Smith, 2022). This evolution is further supported by the development of integrated frameworks that facilitate the seamless application of formal methods throughout the model lifecycle (Laing et al., 2020). These advances open the door to mathematically proving system constraints and safety properties, especially in safety-critical domains, and underscore the growing importance of formal verification in next-generation MBSE toolchains.

Best practices recommend regular application of both manual and automated validation methods throughout the project lifecycle, leveraging simulation, analysis, and traceability tools to ensure completeness and correctness (Visure Solutions, 2024). The emergence of SysML v2, with its official ANTLR-based grammar, opens new possibilities for automated syntactic validation using standard parsing techniques. While ANTLR-based parsers can efficiently detect syntactic errors, they may overlook semantic inconsistencies or domain-specific rule violations unless complemented by additional semantic analysis layers (Tomassetti, 2024). Existing LLM-based generation approaches generally lack tight integration with such formal validation mechanisms, treating validation, if at all, as a separate post-processing step. Recent advances in tool interoperability and open-source validation frameworks are beginning to address these gaps, enabling more seamless integration of validation engines into automated model generation pipelines (Bajaj et al., 2022). The proposed methodology distinguishes itself by embedding ANTLR-based validation directly into the agent's iterative refinement process, using diagnostic feedback to drive self-correction.

2.3. Agent-based systems for automation and design

Intelligent agents have been successfully applied to a wide range of automated design and code generation tasks (Guan et al., 2024; Jiang et al., 2024; Kim et al., 2024; Lopez et al., 2024). Agent-based systems offer several advantages, including autonomy, adaptability, and the

ability to perform iterative refinement. Among the most prominent architectures, Belief-Desire-Intention (BDI) agents have been applied to complex engineering domains, facilitating goal-driven reasoning and dynamic adaptation in collaborative design environments (SmythOS, 2025a). The integration of artificial intelligence techniques into BDI architectures has further enhanced agents' ability to learn, reason, and adapt to complex environments, while preserving transparency and verifiability critical for engineering applications (SmythOS, 2025a).

Agent-based modeling (ABM) has become a foundational tool in modern engineering, particularly for simulating and optimizing complex adaptive systems such as traffic networks, swarm robotics, and supply chains (SmythOS, 2025b). In MBSE, ABM enables the visualization and analysis of emergent behaviors arising from the interactions of autonomous system components, supporting early-phase design decisions and system-of-systems analysis (Maheshwari et al., 2015).

Recent research has introduced the concept of Automated Design of Agentic Systems (ADAS), which leverages foundation models and meta-agents to iteratively invent and refine agentic system designs (Hu et al., 2025). This paradigm shift from hand-crafted to automatically learned agent architectures has shown promise in producing more effective and adaptable agent-based solutions, with meta-agents capable of discovering novel building blocks and workflows that outperform manually designed counterparts.

In the context of code generation, multi-agent frameworks such as AgentCoder have demonstrated that collaboration among specialized agents-such as programmer, test designer, and test executor agents-can significantly improve the correctness and robustness of generated code through iterative self-improvement and feedback loops (Huang et al., 2024). These approaches highlight the value of modularity and division of labor within agentic systems, enabling scalable and resilient solutions for complex automation tasks.

Despite these advances, the application of agent-based approaches to SysML model generation remains relatively unexplored. While LLM-based methods can automate aspects of model synthesis, they often struggle with syntactic correctness and semantic consistency. Traditional validation methods may be manual and insufficient for detecting all error types. Embedding agent-based reasoning and iterative validation directly into the model generation workflow, as proposed in this work, represents a promising direction for achieving robust automation in MBSE.

2.4. Retrieval-Augmented Generation (RAG)

RAG has emerged as a foundational technique to enhance the factual accuracy, relevance, and adaptability of LLMs outputs across a wide range of domains (Zeng et al., 2024; Goover, 2025; Cibrián et al., 2023). RAG architectures combine the generative capabilities of LLMs with retrieval capabilities from external knowledge sources, such as curated document repositories, codebases, or structured databases. Rather than relying solely on the model's internal knowledge which may be outdated, generic, or insufficient for specialized tasks, the system first retrieves contextually relevant documents or data snippets based on the input query. This retrieved information is then incorporated into the prompt, effectively grounding the LLM's output in up-to-date and domain-specific knowledge (CelerData, 2025).

The RAG paradigm has demonstrated significant success in natural language processing (NLP) tasks, including open-domain question answering, content generation, and technical support. In code generation scenarios, RAG enables the retrieval of pertinent API documentation, code snippets, and best practice guidelines, which can be dynamically injected into the LLM's context to guide synthesis and reduce hallucinations (Chitika, 2025). The practical implementation of RAG systems for code or model generation typically involves several key components: a well-structured knowledge base, high-quality embedding models for semantic retrieval, efficient vector databases (e.g., FAISS, Pinecone), and robust retrieval pipelines (Chitika, 2025; CelerData, 2025). Fine-tuning embedding models on domain-specific data and employing prompt engineering strategies further improve retrieval precision and output quality.

Recent advances have also introduced hybrid retrieval models that combine keyword-based and semantic search, as well as multimodal RAG systems capable of integrating textual, visual, and structured data sources (Signity Solutions, 2025; Goover, 2025). These innovations enable AI systems to deliver more context-aware and accurate responses, especially in technical domains where precision is critical. The integration of real-time data feeds allows RAG systems to dynamically update their knowledge base, addressing the challenge of knowledge obsolescence in rapidly evolving fields (Signity Solutions, 2025).

An important trend in RAG research is the addition of interactive feedback mechanisms. By leveraging user or developer feedback on retrieved content or generated outputs, RAG systems can iteratively refine both their retrieval and generation processes, leading to improved factual consistency and structural quality in outputs (Liu et al., 2024b). This is particularly valuable in domains such as code and model generation, where iterative refinement and validation are essential for achieving correctness.

In the context of this work, the RAG paradigm is adapted to the generation of SysML v2 models. The system retrieves semantically similar SysML v2 model examples-including both code and natural language descriptions-from a curated local repository based on the user's prompt. These retrieved exemplars are incorporated into the LLM's prompt, providing structural and syntactic patterns that ground the generation process. This targeted retrieval, combined with an iterative validation loop, distinguishes the proposed methodology from generic code generation approaches, resulting in higher structural correctness and domain relevance from the initial generation step.

Overall, the integration of RAG with interactive feedback and validation mechanisms represents a robust approach to overcoming the limitations of standalone LLMs, particularly in specialized engineering domains where accuracy, traceability, and adaptability are paramount (Goover, 2025; Liu et al., 2024b).

2.5. Positioning of this work

According to the existing works, several persistent limitations in the current landscape of automated SysML v2 model generation and validation in industrial contexts can be found. First, while LLM-based approaches have demonstrated excellent capabilities in code and model synthesis, they frequently lack mechanisms to ensure syntactic validity and semantic coherence, particularly for formal languages such as SysML v2 (Wang et al., 2023b; Zhu et al., 2022; Bonner et al., 2024). Second, conventional MBSE validation practices remain largely manual, tool-specific, or decoupled from automated generation workflows, resulting in fragmented processes that do not allow scalability and reliability (Smith et al., 2023; Bajaj et al., 2022; Hecht and Chen, 2022). Third, although agent-based systems offer autonomy and iterative refinement, their application to the tightly integrated, closed-loop generation and validation of SysML v2 models is still nascent (Guan et al., 2024; Garcia et al., 2022; Hu et al., 2025). Fourth, standard RAG frameworks are predominantly optimized for unstructured textual retrieval, whereas the demands of formal modeling require the retrieval and contextual integration of structured model exemplars (Zeng et al., 2024; Chitika, 2025).

As result of this review, an approach that integrates unified LLM-driven agentic reasoning, example-based retrieval, and formal syntactic validation within an iterative, self-correcting loop appears reasonable for SysML v2 model synthesis. This work addresses these gaps by introducing a framework that integrates: (1) an LLM-powered agent capable of orchestrating multi-step reasoning and tool invocation; (2) a RAG component that retrieves and injects semantically relevant SysML v2 code and descriptions from a curated repository to ground generation in authentic modeling patterns; and (3) a formal validation engine,

Table 1
Capability comparison between existing approaches and proposed methodology.

Aspect	Existing approaches	Proposed methodology
Syntactic validity	LLMs generate models with structural errors (Zhu et al., 2022; Bonner et al., 2024)	ANTLR-based validation ensures 100% syntactic compliance (Parr, 2013)
Semantic grounding	Manual example retrieval or limited RAG implementations (Patel et al., 2024)	Structured model retrieval via domain-specific RAG (Lewis et al., 2020)
Validation integration	Post-hoc validation in separate tools (Bajaj et al., 2022)	Embedded validation loop with auto-correction (Luo et al., 2025)
Formal language support	Generic code generation focus (Ghaemi et al., 2024)	SysML v2 specialization with grammar-aware synthesis
Complexity handling	Limited to component-level modeling (Brown et al., 2021)	Supports system-of-systems decomposition

leveraging the official SysML v2 ANTLR grammar, to provide precise diagnostic feedback and drive iterative self-correction. By embedding these components within a unified, automatic refinement loop, the proposed methodology establishes a robust pipeline for translating natural language requirements into formally valid and semantically meaningful SysML v2 models. This comprehensive integration not only advances the state of the art in LLM-assisted MBSE but also lays the groundwork for scalable, trustworthy, and expert-system-driven model engineering workflows.

Recent approaches to automated model generation exhibit distinct capabilities and limitations, as synthesized in Table 1. While existing methods address aspects of the model synthesis challenge, critical gaps persist in formal language compliance and integrated workflows.

The proposed methodology addresses three fundamental limitations of current systems: (1) the syntactic-semantic disconnect in LLM-generated models (Wang et al., 2023b), (2) the workflow fragmentation between generation and validation (Jones et al., 2019), and (3) the lack of structural exemplars in RAG implementations (Zeng et al., 2024). By integrating ANTLR-based validation directly into the agent's refinement loop and leveraging structured model retrieval, our approach achieves closed-loop generation that maintains formal correctness while preserving semantic intent. This represents a significant advancement over template-based SysML generation (Object Management Group, 2023) and isolated validation tools (Nguyen et al., 2022), establishing a new paradigm for AI-assisted MBSE workflows.

3. Methodology: Agent-based generation with iterative validation

This section details the methodological framework established for the automated transformation of natural language requirements into formally valid SysML v2 models. The approach is designed to address the challenges inherent in bridging informal user intent with the syntactic and semantic validation required by MBSE, ensuring both structural correctness and domain relevance in the generated models.

The presented approach integrates existing natural language processing techniques with domain-specific validation mechanisms to create an agent that facilitates model generation while maintaining compliance with the SysML v2 standard. This process is designed not only to capture the intent expressed in natural language, but also to iteratively refine and validate the output against formal modeling constraints.

In the following subsections, each component of the proposed system, the flow of data through the agent, and the strategies designed to ensure robustness, accuracy, and traceability throughout the transformation process are described (see Fig. 1 for an overview of the system architecture).

3.1. Theoretical foundations

At a theoretical level, the agent developed in this work can be characterized as a goal-driven, modular architecture that bridges informal requirements with formally valid SysML v2 representations. This agent integrates symbolic reasoning, generative synthesis, and iterative correction within a closed-loop control framework governed by linguistic input and formal validation constraints.

The Agent's core functionality can be defined as a 5-tuple (Russell and Norvig, 2021):

$$\text{Agent} = (P, G, A, R, V)$$

where:

- P represents the perception module, responsible for interpreting natural language inputs and retrieving semantically similar SysML v2 examples.
- G denotes the generative process that synthesizes initial candidate models based on the user input and contextual examples.
- A comprises the set of executable actions or tools available to the agent.
- R encompasses the reasoning mechanism that governs multi-step planning and refinement.
- V refers to the validation engine that assesses both syntactic and semantic correctness.

The agent architecture integrates retrieval-augmented semantic grounding with rule-constrained generation, operating under a hypothesis-driven paradigm where each output undergoes systematic verification and correction to ensure convergence toward valid models. This design aligns with principles of Belief-Desire-Intention (BDI) models, which have been widely adopted in agent-oriented programming for their ability to support goal-driven reasoning and dynamic adaptation in complex engineering tasks (Meneguzzi and De Silva, 2015). Recent work in expert systems highlights the effectiveness of BDI architectures for orchestrating automated workflows in domains requiring both robust reasoning and formal validation, as demonstrated in the integration of multi-agent frameworks for intelligent model synthesis and code generation (Chen et al., 2024). In this framework, beliefs represent the agent's synthesized knowledge state, integrating user inputs with context retrieved from domain-specific exemplars (Alelaimat et al., 2023); desires define the target system state as a formally valid SysML v2 model that precisely implements the input requirements; and intentions manifest as the orchestrated sequence of tool invocations and reasoning steps executed to achieve modeling objectives. This BDI-aligned structure enables iterative refinement while maintaining formal constraints, a critical capability for expert systems operating in rigorous engineering domains.

Concretely, each element of the BDI scheme is materialized through the task-specific engines introduced later in Sections 3.3, 3.4. Beliefs are updated continuously by the Validation Engine, whose diagnostic report enriches the agent's knowledge state with precise parse-tree locations and error types, and by the perception layer that retrieves context examples. Desires, the target of delivering a formally correct SysML v2 model, drive the reasoning module to keep the validation status in its goal stack. Intentions become explicit through the ordered invocation of (1) the SysML Generator to enact the creation of SysML

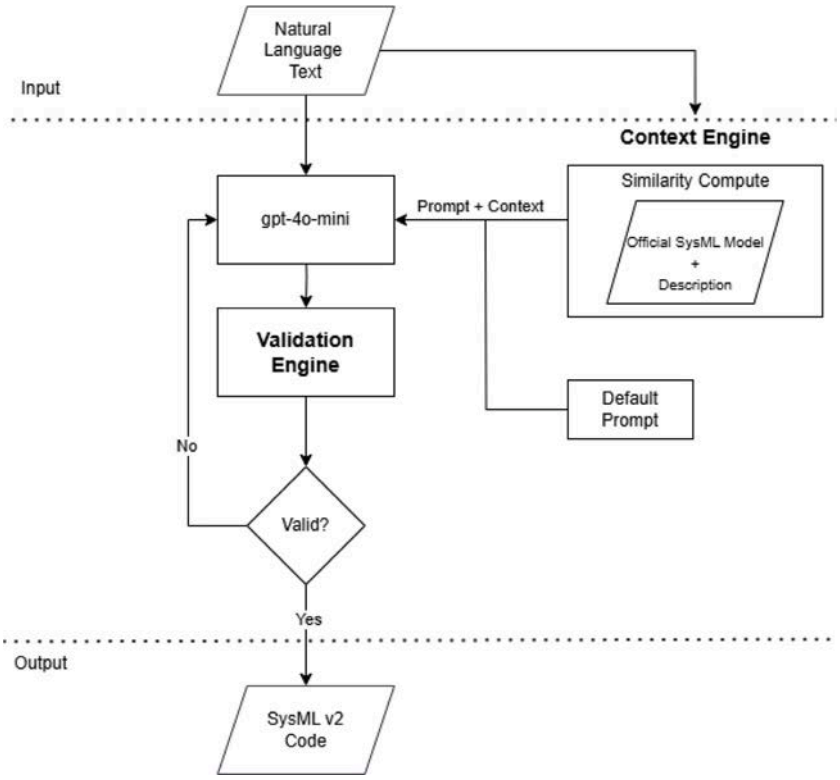

Fig. 1. Architecture of the agent methodology overview.

v2 models, and (2) the SYsML Fixer to execute local repairs when the validator detects errors (Section 3.2.1). Together these engines implement the tuple seen before: the SysML Generator provides G, the Validation Engine provides V and the perceptual feedback for P, while the SysML Fixer, invoked by the ReAct loop shown in Algorithm 1, constitutes the concrete A chosen by the reasoning component R to satisfy the current intention. This mapping turns the abstract BDI cycle into an operational, closed-loop workflow that iteratively converges on a model state where the agent's desires are fulfilled and no further validation errors are observed.

By combining symbolic representations, formal grammar-based validation, and contextualized generative capabilities, the agent establishes a hybrid reasoning workflow. This framework bridges the gap between unstructured natural language and the structural rigor demanded by MBSE practices, positioning the agent as a reliable intermediary in the automated synthesis of SysML v2 artifacts.

3.2. SysML v2 Agent

At the core of the proposed architecture is an intelligent agent, implemented using the Langchain framework (LangChain Team, 2024), which orchestrates the transformation of natural language descriptions into valid SysML v2 models through multi-step reasoning and validation. The agent integrates multiple tools and capabilities, such as conversational memory, context-based prompting, syntax validation, and iterative refinement, to ensure robust and valid output. The memory module, implemented via ConversationBufferMemory, retains the history of user-agent interaction, enabling the agent to incorporate conversational context into its reasoning. This is essential for leveraging feedback from the Validation Engine, which provides detailed error reports including specific locations of detected issues.

The agent leverages OpenAI's GPT-4o-mini through the OpenAI API (OpenAI, 2024), providing advanced natural language understanding and generation capabilities. To further enhance contextual relevance, the agent incorporates a pretrained embedding model (Sentence-Transformers Reimers and Gurevych, 2019) for semantic retrieval of similar SysML v2 examples from a curated local database, following RAG best practices (Lewis et al., 2020).

3.2.1. Toolset

The agent's workflow is modularized through three custom capabilities:

- **SysML Generator:** this module transforms a natural language prompt into SysML v2 code, combining user input with the k most semantically similar examples retrieved from the local database to improve generation relevance.
- **Syntax Validator:** this module parses and validates the generated SysML v2 code against the official standard, returning diagnostic feedback if errors are present.
- **SysML Fixer:** this module applies corrections to the generated SysML v2 code based on the errors identified by the Syntax Validator, iteratively refining the model until it satisfies all syntactic constraints.

3.2.2. Prompting strategy

A key element of agent performance lies in the design of its prompting strategy (Stahl et al., 2024). The system prompt is carefully engineered to guide the language model's behavior toward producing high-quality and valid SysML v2 code. Rather than relying solely on direct user input, the prompt provides a clear role and goal for the agent: to generate correct SysML v2 models and ensure their validity through iterative refinement. The system prompt instructs the agent to: (1) use the generation tool to produce the code, (2) validate the output using a dedicated validator, and (3) analyze the report of the errors provided by the Validator and regenerate until no syntax issues remain.

In this work, the generation of SysML v2 models is structured through a hybrid prompting strategy that leverages both system-level instructions and example-based grounding. This strategy involves two primary prompts: a system prompt and a user prompt, both carefully designed to guide the model's behavior. The system prompt sets the

context for the model, informing it that its role is to assist in creating SysML v2 models based on user input, while the user prompt ensures that the model focuses on delivering accurate and valid SysML v2 code. To achieve this, the full prompt combines both instructions and contextual examples, ensuring that the generated model meets the user's expectations.

- System Prompt: *you are a helpful assistant who is in charge of creating SysML v2 models given an input from a user.*
- User Prompt: *given that user input, give me a valid SysML v2 model in that represents what the user wants. Return ONLY the SysML v2 code. In order to help you, I have extracted the two nearest models that we have in our local database to give you a little bit more of context. Remember, return me JUST the SysML v2 code generated.*

The complete prompt, therefore, consists of both the system-level instructions to set the agent's role and the user-level instructions, along with contextual examples from the local database. This combination of structured guidance and relevant context helps the agent to generate accurate, contextually appropriate SysML v2 code, while also allowing for iterative validation and refinement of the output.

This approach is aligned with recent best practices in prompt engineering, which emphasize the use of system-level instructions and feedback loops to improve the accuracy and reliability of generative AI outputs (Boonstra, 2024). By embedding validation steps directly into the prompting strategy, the agent can not only act reactively to user input but also proactively improve the quality of its generated artifacts. This structured approach promotes self-correction and consistency in output quality. In addition to the system prompt, user queries are enriched with context-relevant examples retrieved from the local database via semantic similarity, forming a hybrid prompting strategy that combines instruction-based guidance with example-based grounding (Brown et al., 2020).

3.2.3. Agent workflow and logic

The agent makes use of the Conversational ReAct architecture, enabling it to plan and execute actions (tool invocations) in a loop based on intermediate observations. The LLM first reasons about the problem and generates a plan of action, then performs the actions in the plan and observes the results. The LLM then uses the observations to update its reasoning and generate a new plan of action. This process continues until the LLM reaches a solution to the problem (Boonstra, 2024). The agent receives a system-level instruction that guides its behavior, enforcing a logic-driven iterative workflow, as shown in Algorithm 1.

Input: User input in natural language
Output: Valid SysML v2 model
begin
  Extract relevant model descriptions from the user input using the **Context Engine**;
  Build a detailed prompt by combining the user input with the extracted contextual descriptions;
  Generate an initial SysML v2 model using the **SysML Generator** with the constructed prompt;
  Validate the generated SysML v2 model using the **Validator Engine**;
  while *Validation result is* ***invalid*** **do**
    Extract syntactic and semantic errors from the validator output;
    Construct a corrective prompt including the current model and the extracted errors;
    Regenerate an improved SysML v2 model using the **SysML Fixer** with the corrective prompt;
    Re-validate the new SysML v2 model using the **Validator Engine**;
  end
  return Validated and correct SysML v2 model;
end

Algorithm 1: Pseudocode of the Agent Workflow.

3.3. Context engine

Following the Agent component, one of the most important and innovative elements of this work is the Context Engine. As its name suggests, this component is responsible for extracting relevant context based on the user's input. In this case, context refers to the k most similar models, based on the similarity between its descriptions and the user's input, retrieved from a local database. This local database contains SysML v2 models, each including a brief description of what the model represents.

3.3.1. Generation of descriptions

The construction of the database is a prerequisite for building the Context Engine. This database was created based on SysML v2 models available in the official documentation (Object Management Group, 2023). Once these models were collected, an expert generated a set of simple descriptions that a user might provide when intending to generate each of the models. All of this information was then stored in a local database.

3.3.2. Embedding similarity

The final step of the Context Engine involves retrieving the k most semantically similar descriptions to a user-provided input. The objective is to identify and extract the models from the local database that are most closely aligned with the user's intent based on their descriptions. Cosine similarity, a metric widely used in NLP for measuring the similarity between two vectors in a high-dimensional space, is employed for this purpose (Farouk, 2019).

In this context, cosine similarity operates not on raw text or tokens, but on their vector representations, known as embeddings. Each textual description must therefore be converted into a fixed-dimensional embedding vector. For this purpose, we utilize the SentenceTransformers library, which offers state-of-the-art models capable of generating high-quality sentence embeddings. SentenceTransformers (UKPLab, 2024) supports a variety of NLP tasks, including semantic similarity, semantic search, and paraphrase mining.

To optimize performance and avoid redundant computations, embeddings for all database descriptions are precomputed and stored locally. The same embedding model is consistently used for both stored descriptions and user input, ensuring vector compatibility and semantic consistency across similarity computations.

Upon receiving a new user input, the system encodes it into an embedding with a real embedding dimension of 384. The cosine similarity with all pre-stored embeddings in the database is then computed. The k descriptions with the highest similarity scores are retrieved, corresponding to existing SysML v2 models in the local repository, and incorporated into the final prompt construction. This context-enriched prompt guides the model generation process, enhancing the relevance and accuracy of the generated SysML v2 artifacts.

To visualize the high-dimensional embeddings, t-SNE (t-Distributed Stochastic Neighbor Embedding) is applied for illustrative purposes. It reduces the 384-dimensional vectors to 2D to support qualitative interpretation of the embedding space (Fig. 2). t-SNE is not used in any part of the retrieval or similarity computation pipeline, which relies strictly on cosine similarity in the original embedding space. The $k = 1$ neighborhood is highlighted for reference.

In summary, this Context Engine enables dynamic, context-aware model generation by leveraging semantic similarity to identify and retrieve the most relevant prior knowledge from a local database. This

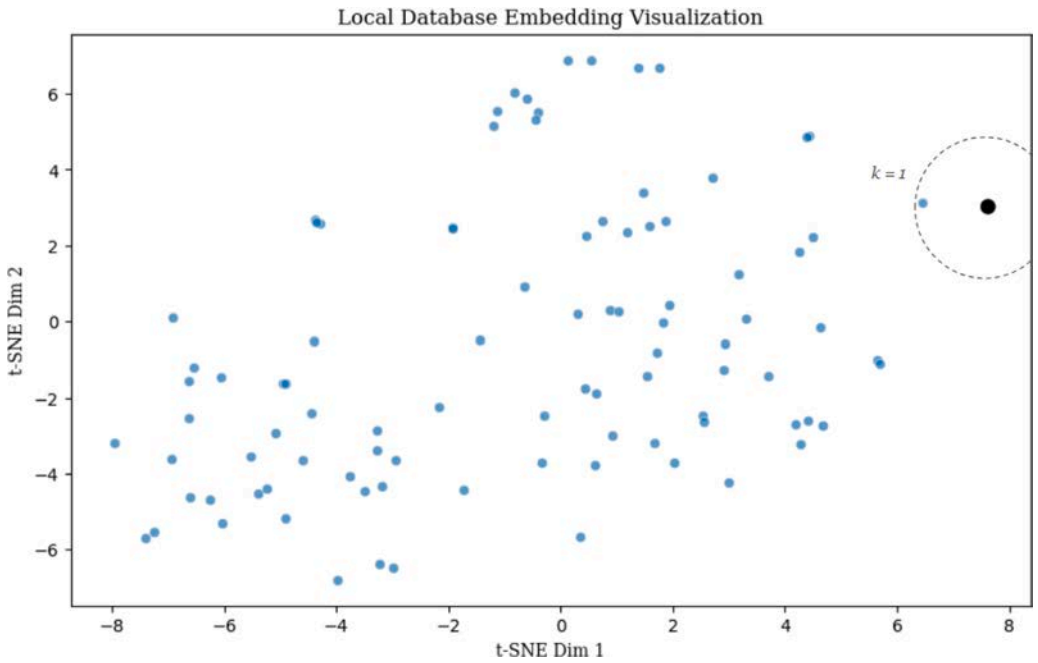

Fig. 2. Visualization of the model's embeddings with k = 1.

retrieval mechanism is important to constructing effective prompts that yield precise and consistent SysML v2 models.

For a real world example, consider a user input requesting: *"Create a model of an energy system with a battery component and a voltage sensor."*. Based on the similarity computation, in the case of $k = 1$, the system retrieves the nearest most relevant model in the database. The model retrieved is: *"Generate a SysML v2 model for an electric vehicle (EV) system focusing on state space representation. Model contains a package named EVSample, which includes parts like Vehicle, Battery, Motor, and Tire. Each part defines its attributes, inputs, outputs, and states using standard units of measure. The Vehicle part integrates mass and dynamics, whereas the Battery part tracks state-of-charge and energy flow."*. As demonstrated, the extracted model provides distinct yet relevant perspectives for designing a system with a battery and sensors. Based on the closest matching description in our dataset in terms of embedding similarity, the corresponding SysML v2 model is retrieved and incorporated into the prompt for the agent. This ensures that the generated SysML v2 model aligns with the user's intent.

3.4. Validation engine

The final module to be addressed is the Validation Engine. As discussed in previous sections, this component is responsible for verifying the syntactic and semantic correctness of the models generated by the agent. It serves as the final checkpoint before a model is returned to the user. If any errors are detected, the agent must interpret the validation feedback and make the necessary corrections to ensure compliance with the defined syntax rules.

The validation of SysML v2 models involves a two-stage process that encompasses both syntactic and semantic analysis. The SysML v2 language specification is defined using a programming-language-agnostic grammar in ANTLR (Parr and Quong, 1995) format, which consists of two sets of rules: lexical rules, which define the basic tokens of the language (such as keywords, identifiers, and symbols), and syntactic rules, which govern the valid structural composition of these tokens.

In the syntactic validation phase, the input model is converted to a character stream, processed by the lexer to generate a sequence of tokens, and then passed to the parser. The parser, configured with the SysML v2 parser grammar, checks whether the token sequence conforms to the syntactic rules and constructs a hierarchical parse tree. A custom error listener detects and reports any syntactic errors, ensuring precise feedback and handling.

Syntactic validation ensures that the input text complies with the grammatical rules defined in the ANTLR grammar (SysML v2 Lexer and Parser). However, syntactic validation alone is not sufficient; semantic validation is required to verify logical consistency and adherence to context-sensitive constraints that cannot be expressed through grammar alone. Examples include ensuring that identifiers are unique within a given scope or that references point to previously defined elements. These validations rely on the structure of the parse tree produced by the syntactic phase, and are applied using ANTLR's listener or visitor mechanisms.

Together, these two phases provide a robust mechanism for validating SysML v2 models. Syntactic validation guarantees structural correctness based on the formal grammar, while semantic validation ensures that the model adheres to the intended semantics of SysML v2, allowing reliable model interpretation and further analysis.

4. Evaluation methodology and results

This section presents the empirical evaluation of the proposed SysML v2 Agent methodology. The primary goal is to assess its effectiveness in generating syntactically valid SysML v2 models compared to baseline LLMs and to understand the contribution of its core components: retrieval-augmented generation and iterative validation.

4.1. Experimental setup

4.1.1. Dataset of prompts

A dataset of $N = 20$ natural language prompts describing SysML v2 modeling tasks was curated for this study. The prompts were systematically designed to ensure comprehensive coverage of the core modeling constructs and use cases illustrated within the official SysML v2 specification and associated examples (Object Management Group, 2023). This included representation of fundamental structural definitions (e.g., parts, interfaces, connections), behavioral specifications

Table 2
Natural language prompts for experimentation.

ID	Description
U_1	Create a part called AvionicMotor. It uses electrical power to generate mechanical torque. It has three properties: rated power in kilowatts, nominal voltage in volts, and maximum speed in RPM.
U_2	Create a model for a drone with a battery, a motor, and an altitude sensor.
U_3	Create a central controller with a temperature sensor that is able to log data.
U_4	Design a robot arm that can rotate 360 degrees and has a gripper to pick up small objects.
U_5	Model a drone system with GPS navigation, a camera module, and an obstacle avoidance function.
U_6	Specify a water pump system that activates below 30% and deactivates at 90% tank level.
U_7	Describe a car that includes an electric motor, battery pack, and regenerative braking.
U_8	Create a coffee machine system with a user interface, water heater, and milk frother.
U_9	Define a process where a user logs into a mobile app and retrieves their profile data.
U_{10}	Model a smart home system that is able to turn on the lights.
U_{11}	Design a patient monitoring device that tracks vital signs and sends alerts.
U_{12}	Define a library management system that is able to borrow and return books.
U_{13}	Model a weather station that collects temperature, humidity, and wind speed data and is able to transmit it.
U_{14}	Create a requirement that an elevator stops at requested floors within 3 s.
U_{15}	Define an interface between a smartphone and a smart TV for screen mirroring functionality.
U_{16}	Model a vending machine that is able to dispose items.
U_{17}	Define the basic structure of a security system with a door sensor and an alarm.
U_{18}	Define a drone that has the ability to pick up objects from a structure.
U_{19}	Define a heating system that adjusts the temperature when there is a trigger.
U_{20}	Create a power unit that supplies electricity to servers and prevents overload.

(e.g., actions, states, transitions), requirements expression and traceability, allocation relationships, and common view/viewpoint definitions pertinent to MBSE workflows.

The set of prompts was further diversified by spanning multiple application domains (including aerospace, automotive, healthcare, smart home, industrial automation, and consumer electronics) to assess the agent's applicability across different contexts. Furthermore, approximately 15% of the prompts were specifically designed to involve concepts or combinations not explicitly present in the examples within the RAG database, thereby providing insights into the agent's generalization capability beyond direct example retrieval. The different prompts used for experimentation can be seen in Table 2. Each prompt was designed such that at least one valid SysML v2 representation exists.

4.1.2. Selection of the different systems under comparison

The following models were selected to compare the different performance for the given problem:

- `SysMLAgent`: The proposed methodology, integrating an LLM (GPT-4o-mini) with RAG using a local database of 92 SysML v2 examples, and the iterative validation loop powered by the ANTLR-based `ValidationEngine`.
- **Baseline LLM 1 (`BL1: GPT-4o-mini`)**: The same base LLM used in `SysMLAgent` (GPT-4o-mini), prompted directly with the natural language input without RAG or iterative validation.
- **Baseline LLM 2 (`BL2: Gemini 2.5 Pro`)**: A second state-of-the-art LLM (Gemini 2.5 Pro), prompted directly.
- **`Agent_NoRAG` (Ablation)**: The agent framework developed in this work with the iterative validation loop but *without* enabling the RAG component (using only the LLM and validator).
- **`LLM_Raw+RAG` (Ablation)**: The base LLM (GPT-4o-mini) prompted directly with the natural language input augmented by the retrieved RAG examples, but without the agent's iterative validation loop.

4.1.3. Configuration

All LLM interactions used a consistent low temperature setting (e.g., T = 0.2) to minimize output randomness. The `ValidationEngine` employs the official SysML v2 ANTLR grammar (Parr, 2013) for syntactic checks.

4.1.4. Evaluation metrics

The systems were evaluated based on the following metrics:

- **Solved prompts (%)** quantifies the system's ability to produce a model output for a given natural language prompt, reflecting the robustness and coverage of the generation process. This metric is essential for benchmarking the practical applicability of automated modeling assistants, as high prompt coverage is a prerequisite for real-world deployment (Morjaria et al., 2024).
- **Syntactic validity rate (%)** measures the proportion of generated models that pass formal validation using the official SysML v2 grammar. This is a critical metric for any system generating formal models, as syntactic correctness is non-negotiable in MBSE workflows and serves as a gating criterion for downstream model use (Sallam et al., 2024). Automated syntactic validation ensures that generated artifacts conform to the structural rules of the target language, reducing the need for manual correction and enabling seamless integration with existing toolchains (Serapio et al., 2024).
- **Manual quality score** (see Table 3.) using a structured rubric is widely recognized as a complement to automated metrics in the evaluation of generative AI systems, especially for tasks involving formal models or code (Galileo AI, 2025; Wu et al., 2023; Wiseman et al., 2020; Amazon Science, 2024). Automated metrics such as BLEU or CodeBLEU focus on surface similarity or functional correctness, but often fail to capture nuanced semantic alignment, completeness, and adherence to user intent (Wu et al., 2023; Wiseman et al., 2020). Human evaluation, by contrast, enables expert annotators to assess whether generated artifacts faithfully represent the requirements, avoid unnecessary complexity, and are free from critical omissions or hallucinations (Galileo AI, 2025; Amazon Science, 2024). The use of a multi-level (e.g., 1–3) scale for semantic fidelity and completeness is consistent with established protocols in data-to-text, code, and model generation, where experts judge outputs as low, partial, or high fidelity with respect to the input specification (Wiseman et al., 2020; Amazon Science, 2024). This approach provides actionable insights into the practical utility and trustworthiness of AI-generated models, especially in domains where correctness and interpretability are paramount.

By combining these metrics, the evaluation framework addresses both reference-based and reference-free assessment paradigms, enabling a nuanced analysis of system strengths and limitations. This

Table 3
Criteria for manual quality assessment of generated SysML v2 models (Scale 1–3).

Score	Description of model quality
1	**Low fidelity/Incorrect**: The generated model exhibits significant semantic misalignment with the prompt. Core requirements specified in the input description are missing, key concepts are incorrectly represented, or the overall structure is fundamentally inconsistent with the requested system view.
2	**Partial fidelity/Incomplete or Noisy**: The generated model addresses the primary requirements of the prompt but suffers from notable omissions (e.g., missing secondary constraints, relationships, or attributes explicitly mentioned or clearly implied) or includes significant extraneous elements not specified nor reasonably inferred from the prompt. While partially capturing the intent, the model lacks completeness or introduces unnecessary complexity/ambiguity.
3	**High fidelity/Accurate and Complete**: The generated model demonstrates high fidelity to the prompt, accurately and completely representing all specified requirements, elements, and relationships within the scope of the input description. The model is semantically coherent and free of superfluous artifacts, constituting a direct and appropriate SysML v2 representation of the user's request.

multi-faceted approach is particularly pertinent in MBSE, where both formal validity and semantic appropriateness are essential for model acceptance and deployment.

4.2. Results

The primary performance comparison across all evaluated systems is summarized in Table 4.

The baseline systems, `BL1` (GPT-4o-mini) and `BL2` (Gemini 2.5 Pro), achieved complete prompt coverage but generated models with 0% syntactic validity and a low average manual quality score (1.0). These results illustrate the fundamental limitations of relying solely on LLMs for formal model generation: without domain-specific validation or correction, the models frequently produce outputs that are neither syntactically valid nor semantically aligned with the prompt (see Fig. 3). For approaches that do not include an iterative validation loop (such as `BL1`, `BL2`, and `LLM_Raw+RAG`), the output is always recorded, regardless of syntactic correctness, so the total number of evaluated outputs is consistently 20. In contrast, agent-based configurations like `Agent_NoRAG`, which include iterative validation, discard invalid models when the agent fails to converge to a syntactically valid solution. Hence, only successfully validated models are considered in the quality scoring. This explains why `Agent_NoRAG` includes fewer than 20 scored instances. Manual quality assessment was performed by a single expert with experience in MBSE and SysML v2, ensuring consistency across tasks. In future work, inter-rater agreement could be explored with multiple evaluators to further strengthen the robustness of manual evaluation. This finding is consistent with prior work highlighting the unreliability of raw LLM outputs in code and model synthesis tasks where formal correctness is required (Morjaria et al., 2024; Sallam et al., 2024).

The `LLM_Raw+RAG` configuration, which augments the LLM with retrieved examples but omits validation, demonstrates a notable improvement, achieving 55% syntactic validity and an average manual quality score of 1.9. This result supports the hypothesis that retrieval-augmented generation can effectively guide LLMs toward producing more structurally and semantically appropriate outputs by providing concrete, contextually relevant exemplars. However, the absence of a validation loop means that only a subset of prompts-typically the less complex ones-are solved correctly, while more challenging cases remain unaddressed.

The `Agent_NoRAG` variant, which applies iterative validation without retrieval, produces valid outputs for only 20% of prompts, with a manual quality score of 1.0. This outcome underscores the critical dependency of the agent's refinement loop on the initial candidate's proximity to a valid solution: without relevant examples to inform the initial generation, the agent often fails to converge on a correct model, even with multiple correction attempts. This phenomenon is well-documented in neural program synthesis literature, where minor errors are tractable but major structural flaws require substantial re-generation (Chen et al., 2021).

The full `SysMLAgent` system, integrating both retrieval-based context enrichment and iterative validation, achieves perfect syntactic validity and prompt coverage (100% for both), along with a high average manual quality score of 2.5. This result demonstrates the effectiveness of combining retrieval and validation: retrieved examples provide structural and semantic templates that ground the LLM's output, while the validation loop ensures formal compliance through iterative correction. The synergy between these components enables robust and accurate SysML v2 model generation from natural language inputs, addressing the key shortcomings of LLM-only approaches.

Error analysis further reveals that baseline models, lacking validation or contextual grounding, frequently generate constructs absent from the official SysML v2 grammar (Object Management Group, 2023), leading to parsing failures. Common issues include hallucinated elements, misuse of reserved keywords, and syntactic misplacements. The integration of a retrieval mechanism substantially mitigates these errors by anchoring the LLM's output to valid SysML v2 examples, which serve as implicit templates for both structure and semantics.

Beyond syntactic failures, a closer inspection of the manual quality scores highlights cases where models, although syntactically valid, suffer from semantic shortcomings. Prompts that combined multiple domain-specific elements (e.g., "a power unit that prevents overload and supplies electricity to servers") or implied non-trivial constraints (e.g., safety timing requirements) often led to incomplete or oversimplified models. These outputs typically lacked essential dependencies, allocation relations, or behavioral logic, indicating that the current system struggles to infer implicit modeling requirements from natural language alone. This underscores the importance of enhancing the semantic grounding capabilities of the system, potentially by expanding the RAG database with more behaviorally rich exemplars and incorporating reasoning mechanisms capable of capturing implicit intent.

In summary, these results substantiate that the combination of retrieval-augmented prompting and iterative validation constitutes a robust framework for generating formally valid and semantically consistent SysML v2 models. This approach addresses critical limitations of existing LLM-only methods and demonstrates strong potential for practical deployment in model-driven engineering workflows where correctness and reliability are paramount.

4.3. Discussion

The evaluation in this work focuses exclusively on LLM-based variants, as the central objective is to explore and improve the capabilities of generative language models for automated SysML v2 model synthesis. While we acknowledge the value of comparing against traditional rule-based or template-driven approaches, such comparisons are not included here for two key reasons. First, most existing tools in this category rely on structured inputs (e.g., predefined forms, logical expressions, or system templates) rather than free-form natural language, making direct comparison with our unstructured-input pipeline methodologically inappropriate. Second, many rule-based tools are domain-specific, proprietary, or do not support the latest SysML v2 standard, limiting their relevance and reproducibility. For these reasons, this study concentrates on analyzing and benchmarking LLM-driven methods exclusively. Future work could explore hybrid approaches or controlled comparisons where input modalities and output formats can be reasonably aligned.

The results provide strong evidence for the effectiveness of the proposed `SysMLAgent` methodology in the automated generation of

Table 4
Syntactic validity and prompt success rate per system.

System	Solved (%) (Prompts)	Syntactic validity Rate (%)	Manual quality (Avg. ± Std) (1–3) Avg. ± (0–1) Std.
`SysMLAgent` (GPT-4o-mini)	100	100	2.5 ± 0.3
BL1: GPT-4o-mini	100	0	1 ± 0
BL2: Gemini 2.5 Pro	100	0	1 ± 0
`Agent_NoRAG` (Abl.)	20	20	1 ± 0
`LLM_Raw+RAG` (Abl.)	100	55	1.9 ± 0.4

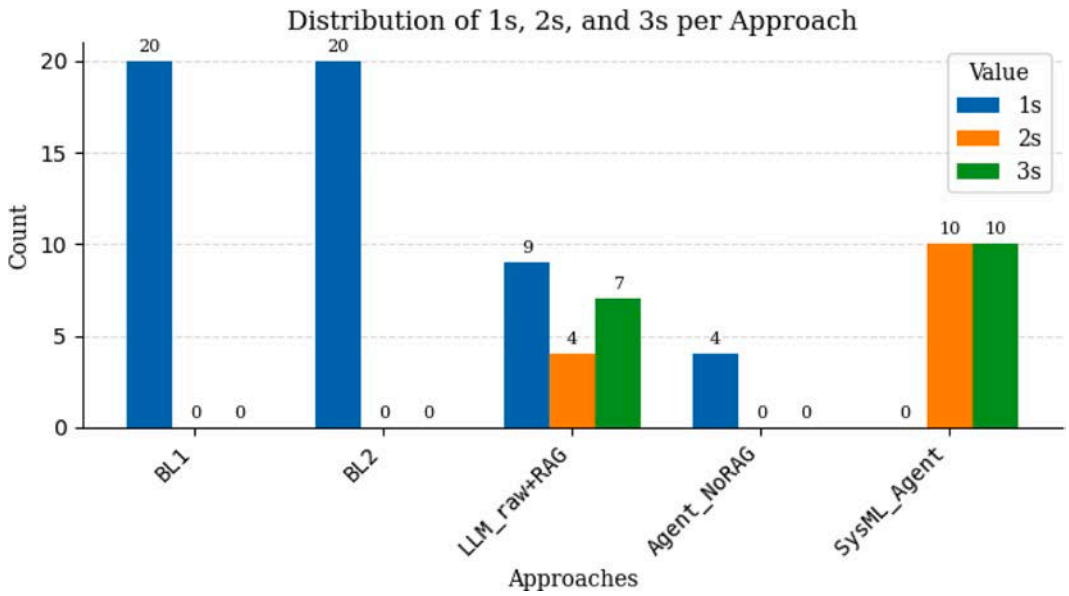

Fig. 3. Manual quality results.

valid SysML v2 models from natural language. The integration of an iterative validation loop is shown to be essential, as it systematically ensures syntactic correctness-a critical requirement for formal modeling languages. The high failure rates of the baseline systems (`BL1` and `BL2`), which lack such a mechanism, highlight the inherent limitations of relying solely on LLMs for this task. This aligns with recent literature underscoring the unreliability of raw LLM outputs in domains requiring strict formalism (Morjaria et al., 2024; Sallam et al., 2024).

Beyond validation, the RAG component emerges as a key contributor to model quality. The performance of the `LLM_Raw+RAG` configuration, which achieves a substantial increase in syntactic validity and manual quality score compared to LLM-only baselines, demonstrates that providing the LLM with relevant, curated examples significantly enhances its ability to internalize and reproduce the structural and semantic conventions of SysML v2. These results indicate that RAG not only improves the plausibility of generated outputs but also acts as a form of implicit domain adaptation, supplying the model with concrete patterns and reducing the likelihood of hallucinated or structurally invalid constructs. Furthermore, when RAG is omitted, as in the `Agent_NoRAG` ablation, the agent's iterative correction mechanism is frequently unable to recover from poor initial generations, underscoring a critical dependency on the quality of the starting point.

The ablation study thus reveals that neither retrieval nor validation alone suffices for robust model synthesis. The validator is effective only when the initial candidate is within a syntactic and semantic neighborhood of correctness; otherwise, even multiple correction attempts cannot repair deeply flawed outputs. This observation is consistent with findings from neural program synthesis, where minor errors are tractable but major structural flaws often necessitate complete re-generation (Chen et al., 2021).

The full `SysMLAgent` system, which combines retrieval-based context enrichment and iterative validation, is the only configuration to achieve perfect results across all evaluation metrics, including solved prompts, syntactic validity, and semantic alignment. This dual-scaffolding architecture enables robust and reliable translation from informal requirements to formal SysML v2 artifacts, demonstrating the practical viability of LLM-driven MBSE workflows when augmented with domain-specific scaffolding.

The implications of these findings are particularly significant for safety-critical domains such as aerospace, automotive, and healthcare, where the correctness and consistency of system models directly impact operational safety and regulatory compliance. In such contexts, syntactic validity alone is insufficient; semantic fidelity and functional correctness are essential to mitigate risks associated with ambiguous or incorrect model artifacts. The methodology proposed here, which combines grammar-based validation with retrieval-informed generation, establishes a foundation for trustworthy AI integration in engineering workflows, addressing both ethical and technical requirements for model integrity.

More broadly, these results suggest that LLMs, when properly structured with domain-specific retrieval and formal validation, can be leveraged as intelligent modeling assistants capable of supporting complex synthesis tasks in formal languages. The demonstrated framework exemplifies how the limitations of general-purpose language models can be overcome through targeted augmentation and iterative refinement, paving the way for scalable and dependable automation in MBSE and related fields.

While this work focuses on ensuring syntactic correctness through formal grammar-based validation, the enforcement of deeper semantic constraints, such as domain-specific rules, physical compatibility between model elements, or behavioral coherence, is currently out of scope. These aspects are essential for guaranteeing full semantic validity in MBSE but require more expressive, context-aware mechanisms beyond ANTLR-based parsing. As future work, we plan to integrate

advanced semantic validation layers, including ontology-driven reasoning and domain-specific consistency checking, to further enhance the correctness and trustworthiness of the generated models.

Despite the promising results, the proposed approach has several limitations that should be acknowledged. First, the system's performance heavily depends on the quality and diversity of the examples stored in the RAG database. While retrieval-based prompting enhances generation, it may struggle with prompts that reference novel configurations or domain-specific constructs not represented in the repository. Second, although the current validation engine ensures syntactic compliance and applies a subset of semantic checks, it does not yet capture high-level domain-specific semantics, behavioral correctness, or system-level constraints. Third, the manual evaluation, while consistent, was conducted by a single expert, which may introduce subjective bias. Future evaluations should include multiple annotators and inter-rater agreement metrics. Lastly, the study is conducted in an offline, controlled environment using a curated prompt set; the scalability and responsiveness of the system under real-time industrial conditions remains to be validated. Addressing these limitations will be essential to move from proof-of-concept to production-ready MBSE toolchains.

5. Conclusions and future work

This work presents a comprehensive evaluation of an agent-based methodology for generating SysML v2 models from natural language prompts using LLMs in industrial contexts. While the dataset used in this study (N = 20) was carefully curated to ensure coverage across a range of modeling scenarios, its limited size poses a constraint on the broader generalizability of the findings. Future work should include scaling the evaluation to a larger and more diverse set of prompts to assess the robustness of the approach across varying domains, complexity levels, and linguistic patterns.

A key insight from this study is that the synergy between RAG and validation is not merely additive, but fundamentally complementary. Retrieval provides the LLM with structurally correct and domain-relevant examples that shape the initial generation, while the validation loop enforces correctness through iterative refinement. Together, these components address the core limitations of standalone LLM outputs, which often include hallucinated syntax or misused domain constructs. The validation framework developed as part of this methodology plays a critical role in ensuring syntactic and semantic correctness. By leveraging ANTLR-based parsing, the system guarantees that generated models conform to the official SysML v2 grammar.

The retrieval of real-world SysML v2 examples acts as a form of implicit supervision. These examples provide the LLM with a structural and lexical blueprint, reducing the likelihood of hallucinated constructs and guiding the model toward valid compositions of elements. This grounding effect is essential for domain-specific modeling tasks, where precise adherence to formal constructs is necessary.

Furthermore, the architecture presented is modular and extensible. Its components, retrieval, validation, and orchestration, are not specific to SysML v2 and could be adapted to other formal modeling languages. This makes the system a generalizable framework for LLM-based generation of formal models in a variety of engineering contexts.

While the proposed agent-based system has been evaluated in controlled offline conditions, its deployment in real-time industrial environments has not yet been tested. Nevertheless, the architecture is modular and compatible with real-time integration scenarios. Given that inference from the selected LLM (e.g., GPT-4o-mini) and the retrieval-validation loop introduce measurable latency, practical deployment would require optimization strategies, such as local hosting of models, batching of requests, or streamlining of validation steps. Preliminary timing analysis indicates that most complete iterations remain within a range acceptable for near-real-time engineering tasks (on the order of seconds), depending on the hardware and LLM interface used. Future work includes deploying the system in a real MBSE pipeline to evaluate data throughput, responsiveness, and integration with industrial modeling tools.

Finally, this study highlights a critical knowledge gap in current LLMs with respect to the SysML v2 language and the broader MBSE domain. Despite their broad training corpora, today's models lack intrinsic understanding of the syntax, semantics, and design patterns inherent to systems engineering formalisms. This underscores the necessity of domain-specific augmentation strategies, such as the one proposed here, to align general-purpose language models with the rigorous demands of specialized engineering disciplines.

CRediT authorship contribution statement

Eduardo Cibrián: Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Resources, Investigation, Formal analysis, Conceptualization. **Jose Olivert-Iserte:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Conceptualization. **Juan Llorens:** Supervision, Conceptualization. **Jose María Álvarez-Rodríguez:** Writing – original draft, Validation, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is part of the UC3M (University Carlos III of Madrid) research project CICLONES: Collaboration and smart continuous integration in Software and Systems Engineering with MBSE and DevOps.

Data availability

Data will be made available on request.

References

Alelaimat, A., Ghose, A., Dam, H.K., 2023. XPlaM: A toolkit for automating the acquisition of BDI agent-based digital twins of organizations. Comput. Ind. 148, 103947.

Amazon Science, 2024. Structured Human Assessment of Text-to-Image Generative Models. Amazon Science, GenomeBench framework.

Bajaj, M., et al., 2022. Systems modeling language (SysML) v2: Towards a next generation modeling language. INCOSE Int. Symp. 32 (1), 1097–1112.

Bonner, M., et al., 2024. LLM-based automation in systems engineering: A case study. Syst. Eng..

Boonstra, L., 2024. Prompt Engineering. Technical Report, Google.

Brown, T.B., et al., 2020. Language models are few-shot learners. In: Advances in Neural Information Processing Systems. Vol. 33, pp. 1877–1901.

Brown, K., et al., 2021. Model checking in MBSE: A survey. IEEE Trans. Softw. Eng..

CelerData, 2025. Latest developments in retrieval-augmented generation.

Chami, M., Bruel, J.-M., 2018. A Survey on MBSE Adoption Challenges. HAL Open Science.

Chen, M., et al., 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.

Chen, Y., et al., 2024. Neural-symbolic BDI-agent as a multi-context system: A case study with negotiating agent. Expert Syst. Appl. 238, 121656.

Chitika, 2025. RAG for code generation: Automate coding with AI & LLMs.

Cibrián, E., Álvarez-Rodríguez, J.M., Mendieta, R., Llorens, J., 2023. Towards a method to enable the selection of physical models within the systems engineering process: A case study with simulink models. Appl. Sci. 13 (21), 11999.

DeHart, D., et al., 2024. Leveraging LLMs for conversational MBSE. In: INCOSE International Symposium.

Dou, S., et al., 2023. What's wrong with your code generated by large language models? An extensive study. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. EMNLP.

Farouk, M., 2019. Measuring sentences similarity: A survey. Indian J. Sci. Technol..

Galileo AI, 2025. Understanding human evaluation metrics in AI: What they are and why they matter. Available at: https://www.galileo.ai/blog/human-evaluation-metrics-ai.
Garcia, P., Smith, T., Lee, H., 2022. BDI agent architectures for collaborative engineering design: A review and future directions. Expert Syst. Appl. 196, 116573.
Ghaemi, M., et al., 2024. Transformers for code generation: A survey. IEEE Trans. Neural Netw. Learn. Syst..
Goover, 2025. Advancements in retrieval-augmented generation.
Guan, Q., et al., 2024. Intelligent agents for automated design. AI Eng..
Hecht, M., Chen, J., 2022. Verification and validation of SysML models. In: System Engineering Forum.
Hu, S., Lu, C., Clune, J., 2025. Automated design of agentic systems. In: Proceedings of the International Conference on Learning Representations. ICLR.
Huang, X., et al., 2024. AgentCoder: Multi-agent based code generation. In: ICSE 2024.
Jiang, Z., et al., 2024. A survey of large language models as agents. arXiv preprint arXiv:2401.12345.
Jones, A., et al., 2019. Peer reviews in MBSE: Effectiveness and limitations. Syst. Eng..
Kausch, H., Pfeiffer, M., Raco, D., Rumpe, B., Schweiger, A., 2025. Enhancing system model quality: Evaluation of the systems modeling language (SysML)-driven approach in avionics. J. Aerosp. Inf. Syst. 22 (5), 367–378.
Kim, S., et al., 2024. Multi-agent systems for code generation. In: Proceedings of the 2024 ACM Symposium on AI.
Laing, C., David, P., Blanco, E., Dorel, X., 2020. Questioning integration of verification in model-based systems engineering: an industrial perspective. Comput. Ind. 114, 103163.
LangChain Team, 2024. LangChain documentation. Available at: https://python.langchain.com.
Lee, T., et al., 2020. Consistency checking in SysML tools. Softw. Syst. Model..
Lewis, P., et al., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems.
Li, Y., et al., 2022. Competition-level code generation with AlphaCode. Nature 603 (7902), 527–534.
Liu, M., et al., 2024a. Evaluating large language models in class-level code generation. In: Proceedings of the 46th International Conference on Software Engineering. ICSE.
Liu, Y., et al., 2024b. Leveraging LLM and user feedback to improve retrieval-augmented generation when question and answer domains shift. In: Proceedings of the 2024 Annual Meeting of the Association for Computational Linguistics. ACL.
Lopez, M., et al., 2024. Agent-based model synthesis in MBSE. Syst. Eng..
Luo, J., et al., 2025. Large language model agent: A survey on methodology, applications and challenges. arXiv preprint arXiv:2503.21460.
Maheshwari, A., Kenley, C.R., DeLaurentis, D.A., 2015. Creating executable agent-based models using sysml. In: INCOSE International Symposium. pp. 1–15.
Meneguzzi, F., De Silva, L., 2015. Planning in BDI agents: a survey of the integration of planning algorithms and agent reasoning. Knowl. Eng. Rev. 30 (1), 1–44.
Morjaria, L., et al., 2024. Examining the efficacy of ChatGPT in marking short-answer assessments in an undergraduate medical program. Int. Med. Educ. 3 (1), 32–43.
Nguyen, H., et al., 2022. Automated linters for MBSE. J. Syst. Archit..
Object Management Group, 2023. OMG systems modeling language (SysML) v2 release. Available at: https://github.com/Systems-Modeling/SysML-v2-Release.
Ojuri, S., Han, T.A., Chiong, R., Di Stefano, A., 2025. Optimizing text-to-SQL conversion techniques through the integration of intelligent agents and large language models. Inf. Process. Manage. 62 (5), 104136. http://dx.doi.org/10.1016/j.ipm.2025.104136.
OpenAI, 2024. Openai API documentation. Available at: https://platform.openai.com/docs.
Pallagani, V., et al., 2024. Prospects and challenges of LLMs in engineering design automation. In: Design Automation Conference.
Parr, T., 2013. The Definitive ANTLR 4 Reference. The Pragmatic Bookshelf.
Parr, T.J., Quong, R.W., 1995. ANTLR: A predicated-ll(k) parser generator. Softw.: Pr. Exp. 25 (7), 789–810.
Patel, R., et al., 2024. RAG for code generation: Opportunities and challenges. arXiv preprint arXiv:2402.12345.
Reimers, N., Gurevych, I., 2019. Sentence-BERT: Sentence embeddings using siamese BERT-networks. arXiv preprint arXiv:1908.10084.
Russell, S., Norvig, P., 2021. Artificial Intelligence: A Modern Approach, fourth US ed. Pearson.
Sallam, M., Barakat, M., Sallam, M., 2024. METRICS: establishing a preliminary checklist to standardize the design and reporting of generative artificial intelligence-based studies in healthcare education and practice. Interact. J. Med. Res. 54704.
Serapio, A., et al., 2024. An open-source fine-tuned large language model for radiological impression generation: a multi-reader performance study. BMC Med. Imaging 24 (1), 254.
Signity Solutions, 2025. Trends in active retrieval augmented generation: 2025 and beyond.
Smith, J., 2022. Using SysMLv2 and formal methods to prove autonomous systems are safe. In: Auto.AI Conference.
Smith, J., et al., 2023. Automated model validation in MBSE. IEEE Syst. J..
SmythOS, 2025a. Understanding BDI agents in agent-oriented programming. Accessed April 2025.
SmythOS, 2025b. Applications of agent-based modeling. Accessed April 2025.
Stahl, B., et al., 2024. Exploring prompt engineering for LLMs in MBSE. arXiv preprint arXiv:2403.45678.
Sun, Y., et al., 2023. CodePLAN: Solution plan guided code generation with chain-of-thought prompting. In: Proceedings of the 37th AAAI Conference on Artificial Intelligence.
Tomassetti, F., 2024. Best practices for ANTLR parsers.
UKPLab, 2024. SentenceTransformers documentation. Available at: https://www.sbert.net.
Visure Solutions, 2024. Best practices for MBSE.
Wang, Y., et al., 2023b. A review of large language models for code generation. IEEE Trans. Softw. Eng. Early Access.
Wiseman, S., Shieber, S.M., Rush, A.M., 2020. Have your text and use it too! end-to-end neural data-to-text generation with semantic fidelity. In: Proceedings of the 28th International Conference on Computational Linguistics. COLING, pp. 2412–2424.
Wu, Z., Li, W., Zhang, Y., 2023. Evaluation methods for code generation models. In: Proceedings of the 2023 International Conference on Artificial Intelligence and Data Science.
Xu, J., Wang, Y., Li, S., 2023. Ontology-enhanced large language models for code and model generation. IEEE Trans. Knowl. Data Eng. 35 (7), 1234–1248.
Yuan, D., Yang, G., Zhang, T., 2025. UI2HTML: Utilizing LLM agents with chain of thought to convert UI into HTML code. Autom. Softw. Eng. 32 (2), 41. http://dx.doi.org/10.1007/s10515-025-00509-5, Springer.
Zeng, M., et al., 2024. What makes a good retriever for open-domain question answering?Trans. Assoc. Comput. Linguist. 12, 1–19.
Zhu, X., et al., 2022. Model checking for code generation with LLMs. arXiv preprint arXiv:2212.09876.
Ziegler, D.M., et al., 2019. Fine-tuning language models from human preferences. In: Advances in Neural Information Processing Systems.